

CS5960 Project Specification

Santiago Franco

September 23, 2026

This project consists of two parts, both of which must be completed. Additionally, all work must be done **individually**.

Part 1: Tower of Hanoi

Goal: You will reflect upon the search algorithms learned during the lectures and implement two of them to solve the Tower of Hanoi.

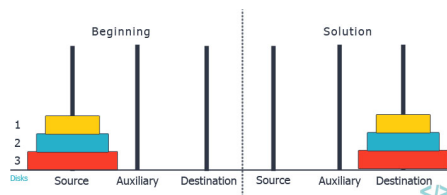


Figure 1: Tower of Hanoi

Setting of the game:

- The Tower of Hanoi is a puzzle game with usually 3 pegs and a fixed number of disks. For this project, we will use **3 pegs**. Consider $n > 0$ number of discs.
- Initial state: At the start, all the disks are in the leftmost peg, with the largest disk on the bottom and the smallest on the top.
- Goal state: All disks are moved over to the rightmost peg.

Rules of the game:

- You can only move one disk to any other peg in each move, and you can only move the top disk on a peg.
- The top disk of any peg can be moved to the top disk of any other peg, with the restriction that you cannot move a larger disk on top of a smaller disk (i.e., disks can only be moved to empty pegs or on top of larger disks).
- This is a unit-cost domain, all moves cost the same. Path cost is its length.

1.1 Core Programing Tasks

[18 marks]

Tasks that you have to implement:

1. Create an efficient representation of the problem to be used with the search algorithms that you have learned in this course.
2. Implement the A* algorithm and another search algorithm of your choice to solve the problem.
3. Create at least two heuristic functions for this problem to work with A*. At least one of them needs to be admissible, and at least one of them needs to be inadmissible.
4. Make sure to include instructions on how to run A* with either of the heuristics and also the other search algorithm. Write this instructions into a file titled "README.txt".

1.2 Extension Programing Tasks

[7 marks]

1. Devise a Pattern Database (PDB) heuristic for the problem. Note that supplementary material is available [here](#).
2. Let m to be the number of disks in your PDB and n be the number of disks in the problem. Implement an iterative function that calculate the biggest m such that constructing the PDB takes less than 5 minutes.
3. Take m from the previous question and increase n , starting from $n = m + 1$ until no solution can be found in less than 10 minutes. Implement an iterative function that sequentially increases n , outputs runtime vs. n , and stops when runtime goes over 10 minutes.
4. Make sure to include instructions on how to run A* with the PDB heuristic and to calculate m and n for steps 2 and 3. Write this instructions into a file titled "README_PDB.txt".

1.3 Core Report Tasks

[18 marks]

The final report should include the following points:

- Explain the rationale behind the representation you have chosen for the problem.
- Briefly describe the search algorithms that you have chosen to solve the problem. Make sure to include whether the algorithm is guaranteed to be optimal, if it is complete and its space and memory $O()$ complexity.
- Describe the two heuristics (called h_1 and h_2 from now on) that you have devised for the problem, explaining why one is admissible while the other is not. You must include whether A* with h_1 and h_2 are guaranteed to provide an optimal solution in your answer.
- Show a table comparing the runtime of A* with h_1 , h_2 and the other algorithm you implemented vs. number of discs n .
- Given a 3-peg and 3-disk Towers of Hanoi like the one in figure 1, draw a search graph for A* (use the out-of-place heuristic), each node must show their g , h , and f values. Secondly, draw a search graph for BFS. Finally, also draw a search graph for DFS. For BFS and DFS make sure to include each node's g value. All nodes must be labeled with their expansion order. You must stop expanding nodes once you either create the goal node or have generated at least 20 nodes.

1.4 Extension Report Tasks

[7 marks]

- Explain the rationale behind the construction of the PDB heuristic.
- Discuss points 2 and 3 of the extension. In particular:
 - Provide the biggest m such that constructing the PDB takes less than 5 minutes.
 - Take m from the previous question and increase n , starting from $n = m + 1$ until no solution can be found in less than 10 minutes. Show how performance worsens as n becomes bigger (runtime vs. n).

Part 2: CSP

2.1 AC-3

Consider the following constraint network: $\gamma = (V, D, C)$:

- Variables: $V = \{a, b, c, d, e, f\}$
- Domains: For all $v \in V : D_v = \{1, 2, 3, 4, 5\}$
- Constraints: $a = e$, $d = e - 2$, $b = a - 1$, $c = d + 3$, $f = b + 3$

1. Run the AC-3 algorithm on the given constraint network. For each iteration of the while-loop, give the content of the data structure M at the start of the iteration, the pair (u, v) removed from M , the domain D_u of u after the call to *Revise* (γ, u, v) , and the pairs (w, u) added into M . Once you are finished running the algorithm, write down the range of valid values for each variable if a solution exists.

Note: Initialise M as a lexicographically ordered list (i.e., (a, b) would be before (a, c) , both before (b, a) etc., if any of those exist). After initialisation, use M as a FIFO queue, i.e., always remove the first (oldest) element from the queue and add new elements to the end of the queue.
[15 marks]

2. Draw the constraint graph of γ . [5 marks]
3. What is the most important advantage of AC-3 vs Acyclic-CG? [5 marks]
4. What is the most important advantage of Acyclic-CG vs AC-3? [5 marks]

2.2 AcyclicCG

Consider the following constraint network: $\delta = (V, D, C)$:

- Variables: $V = \{a, b, c, d, e, f\}$
- Domains: For all $v \in V : D_v = \{1, 2, 3, 4, 5\}$
- Constraints: $a = e$, $d = e - 2$, $b = a - 1$, $c = 2b$, $f = b + 3$

1. Draw the constraint graph of δ . [5 marks]
2. Run the AcyclicCG algorithm on δ . First, pick a as the root and draw the directed tree obtained by performing step 1. Then, give the resulting variable ordering obtained (step 2). Break ties using alphabetical order. List the calls to *Revise* $(\delta, v_{parent(i)}, v_i)$, and, for each of them, give the resulting domain of $v_{parent(i)}$ (step 3). For each recursive call to *BacktrackingWithInference* (step 4) give the domain D'_{v_i} of the selected variable v_i after Forward Checking and the value $d \in D'_{v_i}$ assigned to v_i .
[15 marks]

3. How many times will the AcyclicCG backtrack when solving a CSP problem. Justify your answer in detail. [5 marks]

Part 3: Submission

Submission Instructions

Files:

- For the Programming Tasks in Part 1.1 & 1.2, place your appropriately documented code files in a separate folder called “Source”. You **must** include a “README.txt” file with installation (dependencies) and running instructions.
- For the Report questions in Part 1, create a PDF called “Report.pdf”.
- For the CSP questions in Part 2, create a PDF called “CSP.PDF”.

Instructions:

Deliverables need to be packaged as a single **zip** file and uploaded to Moodle by **December 9th, 2026, no later than 2 pm**. The project is worth 30% of the total score. Make sure to submit to the correct link. Note that you are only allowed to submit to the extension link if you got an official extension approval email from CSM.

- [Regular Submission link](#)
- [Extension link](#)

If you get an extension for this submission, make sure to follow the steps in **Note2** below. The ZIP file must be named according to the following convention:

CS5960_T25-1_FirstName-LastName_Project.zip

where you need to replace the first and last name.

Feedback will be provided on the date written on the coursework grid: <https://moodle.royalholloway.ac.uk/mod/page/view.php?id=862408>.

Note1: We will not accept either handwriting or hand-drawn, please use a suitable drawing tool, e.g. draw toolset in your favorite word processor or [edotor](#) for graphs or tree drawings

Note2: If you get an extension, make sure to let us know via **email** so you can submit in the extension link.

Learning Outcomes

Symbolic AI fundamentals including Problem Modelling, Search Algorithms, Heuristics and Constraint Satisfaction Programming techniques.

Use of AI

You may use generative AI tools in preparing your submission for this assignment for research purposes only, e.g. to complement the slides in terms of information gathering, e.g. how does algorithm "X" work, provide example pseudo-code, etc.

Your submission must include a statement containing the following information.

Name, version (if available), and provider of the generative AI tool used (e.g. Copilot, Microsoft). URL of the tool used (e.g. <https://copilot.microsoft.com>).

A short description of how the generative AI tool was used in the assignment. It is not sufficient to simply say you used the tool, e.g., "for inspiration", "to get started", or "to generate ideas"; you must give a clear description of the input you gave to the tool, the content that it generated, and which parts of your submission are your own original contributions.

More information regarding the use of generative AI in your studies can be found [here](#).

Academic Misconduct

Coursework submissions are routinely checked for academic misconduct (working together, copying from sources, etc.). Penalties can range from a 10% deduction of the assignment mark, zero for the assignment or the case being referred to a Senior Vice-Principal to make a decision (for repeat offences). Note that it is academic misconduct if you fail to correctly reference or acknowledge where you have used generative AI tools to help you produce your submission. If the use of generative AI tools is prohibited for this assignment, then any use of such tools constitutes academic misconduct. Further details can be found [here](#).

Marking Criteria

Full marks will be awarded for correct answers to the code and report questions, each question has the associated score. In general, minor mistakes will cost 10% of the question while major mistakes will result in bigger penalties.

Any questions regarding this assignment can be asked in the labs. Alternatively, you can contact me at santiago.francoaixela@rhul.ac.uk or if you prefer you can also post your question to the module's forum [here](#).

Extensions

This assignment is subject to University policy on extensions. If you believe you require an extension please read the documentation carefully and guidelines [here](#).

Extenuating Circumstances

If you submit an assessment and believe that the standard of your work was substantially affected by your current circumstance then you can apply for Extenuating Circumstances. Details on how to apply for this can be found [here](#). It is essential you read the accompanying documentation on the above link carefully.

Late Submission

In the absence of acceptable extenuating cause, late submission of work will be penalized as follows:

1. for work submitted up to 24 hours late, the mark will be reduced by ten percentage marks;
2. for work submitted more than 24 hours late, the maximum mark will be zero.